

1. THE QUALITY CHALLENGE

Modern software and artificial intelligence (AI) implementations frequently encounter recurring defects, escaped defects, shifting requirements, and failures that surface late in the software development lifecycle (SDLC). Within an enterprise technology-services environment operating at Accenture’s scale, uncovering defects exclusively during final User Acceptance Testing (UAT) or post-deployment significantly inflates remediation costs and jeopardizes overall delivery standards.

“How can defects be identified earlier, their critical root causes isolated, and recurring defects prevented before they reach production?”

Note: This document represents an illustrative Quality Engineering case scenario designed for an enterprise technology-services scale, rather than an internal Accenture defect disclosure.

2. PROPOSED QUALITY ENGINEERING APPROACH

To shift from reactive defect detection to proactive defect prevention, a structured end-to-end TQM methodology is deployed across the project lifecycle:

Problem Identification	Cause Analysis	Pareto Prioritisation	Shainin Red X	PDCA Improvement	Pilot & SPC	Control
Problem Identification Systematically capture and categorize software and AI defects (e.g., hallucination anomalies, prompt regressions, integration failures, API boundary mismatches). Quantify their downstream impact on quality metrics, rework hours, and client experience.						
	Pareto Analysis (80/20 Rule) Analyze historical defect logs to isolate the vital few defect categories contributing to the majority of quality escapes. Pareto prioritization ensures engineering effort is directed where it yields the maximum risk reduction.					
Fishbone / Ishikawa Analysis Organize potential defect drivers into structured operational categories: • Requirements: Ambiguous user stories, unhandled edge cases. • Code: Logic gaps, technical debt, concurrency bugs. • Testing: Inadequate test coverage, stale test data. • Data / Environment: Skewed training data, config drift. • Process / People: Handoff friction, training gaps. <i>Fishbone structures possible causes but requires empirical validation to isolate root causes.</i>						
		Core Diagnostic Distinction To avoid diagnostic ambiguity, the framework enforces a strict separation of analytical tools: • Fishbone: Generates and categorizes <i>possible</i> causes. • Pareto: Prioritizes high-impact defect categories. • Red X: Isolates the exact dominant root cause through experimentation.				

3. SHAININ RED X — CRITICAL CAUSE IDENTIFICATION

Once Pareto analysis identifies the major defect category, the investigation proceeds to Shainin methodology. While Pareto highlights major contributors, complex AI and software environments often suffer from multi-variable interactions. Shainin Red X isolates the dominant root cause (the Red X) through structured comparison, component search, and data-driven experimentation rather than trial-and-error.

4. PDCA — IMPROVEMENT IMPLEMENTATION

The solution is operationalized using the Plan-Do-Check-Act (PDCA) cycle:

Plan & Do

- **PLAN:** Define the critical quality defect, align with suspected Red X findings, design targeted countermeasures, and establish baseline KPIs.
- **DO:** Implement the countermeasure on a controlled project module or AI pipeline component.

Check & Act

- **CHECK:** Compare pre-pilot and post-pilot metrics to validate whether defect occurrence and escape rates have decreased.
- **ACT:** Standardize successful improvements, update automated quality gates, and codify guidelines.

5. PILOT → VALIDATION → CONTROL

Execution follows a rigorous validation and stabilization sequence:

- **Pilot:** Test the countermeasure on a confined service line or AI model iteration prior to enterprise-wide rollout.
- **Validation:** Track key engineering performance indicators: *Defect Density*, *Defect Escape Rate*, *Defect Leakage*, *Reopened Defects*, *Defect Detection Time*, *Automated Test Coverage*, and *Client-Reported Defects*.
- **Monitoring:** Apply Statistical Process Control (SPC) control charts to verify process stability over time.
- **Control:** Institutionalize standard operating procedures and automated guardrails to prevent process regression.

6. COMPLETE QUALITY LOOP

Detect → Analyse → Prioritise → Isolate → Improve → Pilot → Validate → Monitor → Standardise → Continuously Improve

"The objective is not simply to find defects faster, but to move the quality process towards defect prevention and sustained process improvement."

7. EXPECTED QUALITY IMPACT

Operational Outcomes

- Earlier identification of critical quality and AI risks
- Significant reduction in recurring software defects
- Lower production defect escape rates

Strategic Outcomes

- Reduced rework and optimized engineering bandwidth
- Enhanced process stability and data-driven decisions
- Stronger organizational continuous-improvement culture

Disclaimer: This is not an original Accenture project, internal Accenture case study, or representation of Accenture's proprietary quality processes or data. This case study has been independently created by Jagriti Singh as part of a 100 Days TQM & Quality Engineering Challenge to demonstrate the practical application of Quality Engineering and TQM methodologies.